

Dưới đây là hướng dẫn thực tế để thiết lập pipeline CI/CD với GitLab CI cho dự án booking hotel, kèm mẫu .gitlab-ci.yml, cách cấu hình runner, biến môi trường, review apps, caching, artifacts, bảo mật và best practices.

1. Tổng quan kiến trúc CI/CD đề xuất

- Repository: tách thành các project riêng trong Group: backend-api, web-frontend, mobile (nếu cần), infra-terraform.
- Pipeline stages chung: validate → build → test → package → publish (image) → deploy → post-deploy checks.
- Environments: review (ephemeral), staging, production.
- Runners: self-hosted Docker/Kubernetes runners cho hiệu năng và quyền kiểm soát; có runner macos nếu build iOS.
- Secrets: lưu trong GitLab CI/CD Variables (protected + masked) hoặc Vault/Azure Key Vault kết hợp.
- Image registry: GitLab Container Registry hoặc external registry (ECR/GCR).

2. Cấu hình Runner cơ bản

- Chọn executor: docker executor (phổ biến) hoặc kubernetes executor (scale tốt cho nhiều job).
- Cài đặt: cài gitlab-runner, đăng ký runner với token project/group; gán tags (e.g., docker, k8s).
- Security: chạy runner trong container, giới hạn quyền network; set concurrency; dùng non-root user.
- Autoscale: nếu dùng k8s executor, cấu hình autoscaling để tiết kiệm chi phí.

3. Biến CI/CD và quản lý secrets

- Đặt biến ở Group hoặc Project Settings → CI/CD → Variables.
- Đánh dấu: Protected (chỉ cho branches/tags bảo vệ), Masked (ẩn trong logs).
- Ví dụ biến: DOCKER_REGISTRY, DOCKER_USERNAME, DOCKER_PASSWORD (masked); KUBE_CONFIG or KUBE_TOKEN (protected); STRIPE_KEY (masked).
- Tránh commit secrets vào repo; nếu lỡ commit, rotate ngay và scrub history.

4. Chiến lược branch và trigger pipeline

- Branch policy: protected main/master và release/*. Pipelines required on MR → không merge nếu pipeline fail.
- Pipelines per MR: bật pipelines for merge requests; enable Merge Trains (tùy cần).
- Trigger manual deploy jobs for production (manual job + approval).

5. Caching, artifacts và parallel jobs

- Cache: lưu dependencies (node_modules, .m2) giữa jobs để tăng tốc.
- Artifacts: lưu kết quả build (dist, bundles, test reports) để dùng ở job sau hoặc để download.
- Parallel: chia test suite thành nhiều parallel jobs nếu cần (matrix or parallel: <n>).
- Example: cache key: "*CI_PROJECT_ID* - {CI_COMMIT_REF_SLUG}-node-modules".

6. Review apps và Environments

- Review apps: tự động deploy nhánh feature thành ephemeral env để review UI (frontend) hoặc full stack nếu có infra ephemeral.
- Environment naming: review/\${CI_COMMIT_REF_SLUG}; staging; production.
- Cleanup: tự động xóa review app sau merge hoặc sau timeout.

7. Bảo mật pipeline (gating)

- Chạy SAST/Dependency Scanning/Container scan trong pipeline stages (block merge on critical vulns).
- Use only/except or rules để hạn chế job chạy trên branches phù hợp.
- Mask sensitive logs; set job permissions minimal.

8. Logging & Monitoring pipeline

- Kết nối GitLab CI với monitoring: alert khi pipeline fail nhiều lần; thống kê thời gian pipeline trung bình.
- Lưu test reports (JUnit, cobertura) và code quality reports dưới artifacts để GitLab UI hiển thị.

9. Mẫu .gitlab-ci.yml (Node.js backend + React frontend + Docker + Kubernetes)

Tệp mẫu bên dưới là cấu trúc đầy đủ, có stages: validate, install, build, test, image, deploy-review, deploy-staging, deploy-prod. Điều chỉnh image base, scripts và secrets phù hợp môi trường bạn dùng.

```
stages:
  - validate
  - install
  - build
  - test
  - image
  - deploy
  - post-deploy

variables:
  # Registry
  DOCKER_REGISTRY: "$CI_REGISTRY"
  IMAGE_TAG: "$CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA"
  # Cache key pattern (optional)
  CACHE_KEY: "${CI_PROJECT_ID}-${CI_COMMIT_REF_SLUG}"

default:
  image: node:18
  before_script:
    - npm config set progress=false
    - npm ci

.cache_node_modules:
  cache:
    key: "${CACHE_KEY}-node-modules"
    paths:
      - node_modules/

validate:
  stage: validate
  script:
    - echo "Linting YAML and basic checks"
    - npm run lint:js || true
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event" || $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH

install:
  stage: install
  extends: .cache_node_modules
```

```
script:
  - echo "Dependencies installed by default in before_script"
artifacts:
  paths:
    - node_modules/
  expire_in: 1 hour
rules:
  - if: $CI_PIPELINE_SOURCE != "schedule"
```

```
build_frontend:
  stage: build
  image: node:18
  variables:
    NODE_ENV: production
  script:
    - cd frontend
    - npm ci
    - npm run build
  artifacts:
    paths:
      - frontend/build
    expire_in: 1 day
  rules:
    - if: $CI_COMMIT_BRANCH
```

```
build_backend:
  stage: build
  image: node:18
  script:
    - cd backend
    - npm ci
    - npm run build
  artifacts:
    paths:
      - backend/dist
    expire_in: 1 day
  rules:
    - if: $CI_COMMIT_BRANCH
```

```
test:
  stage: test
  image: node:18
  script:
    - cd backend
```

```
- npm ci
- npm run test:ci -- --reporter mocha-junit-reporter --reporter-options
mochaFile=./reports/backend-junit.xml || true
- cd ../frontend
- npm ci
- npm run test:ci -- --ci --reporters=jest-junit || true
artifacts:
  when: always
  paths:
    - backend/reports/
    - frontend/junit.xml
  expire_in: 1 day
rules:
  - if: $CI_PIPELINE_SOURCE

docker_build_push:
  stage: image
  image: docker:24
  services:
    - docker:dind
  variables:
    DOCKER_TLS_CERTDIR: "/certs"
  before_script:
    - echo "$CI_REGISTRY_PASSWORD" | docker login -u "$CI_REGISTRY_USER" --
password-stdin $DOCKER_REGISTRY
  script:
    - docker build -t $IMAGE_TAG -f backend/Dockerfile .
    - docker push $IMAGE_TAG
  only:
    - branches
  rules:
    - if: $CI_COMMIT_BRANCH

deploy_review:
  stage: deploy
  image: bitnami/kubectl:latest
  script:
    - echo "Deploying review app to k8s"
    - export REVIEW_NAME=review-$CI_COMMIT_SHORT_SHA
    - kubectl --kubeconfig="$KUBE_CONFIG" apply -f infra/k8s/backend-deployment-
review.yaml
  environment:
    name: review/$CI_COMMIT_REF_SLUG
    url: https://$CI_COMMIT_REF_SLUG.review.example.com
```

```
on_stop: stop_review
rules:
  - if: $CI_PIPELINE_SOURCE == "merge_request_event"

stop_review:
  stage: deploy
  image: bitnami/kubectl:latest
  script:
    - export REVIEW_NAME=review-$CI_COMMIT_SHORT_SHA
    - kubectl --kubeconfig="$KUBE_CONFIG" delete -f infra/k8s/backend-deployment-
review.yaml || true
  when: manual
  environment:
    name: review/$CI_COMMIT_REF_SLUG
    action: stop
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"

deploy_staging:
  stage: deploy
  image: bitnami/kubectl:latest
  before_script:
    - echo "$KUBE_CONFIG_STAGING" | base64 -d > kubeconfig_staging
    - export KUBECONFIG=$PWD/kubeconfig_staging
  script:
    - helm upgrade --install booking-backend infra/helm/backend --namespace staging --
set image.tag=$CI_COMMIT_SHORT_SHA
  environment:
    name: staging
    url: https://staging.example.com
  rules:
    - if: $CI_COMMIT_BRANCH == "develop"

deploy_production:
  stage: deploy
  image: bitnami/kubectl:latest
  before_script:
    - echo "$KUBE_CONFIG_PROD" | base64 -d > kubeconfig_prod
    - export KUBECONFIG=$PWD/kubeconfig_prod
  script:
    - helm upgrade --install booking-backend infra/helm/backend --namespace
production --set image.tag=$CI_COMMIT_SHORT_SHA
  environment:
    name: production
```

```
url: https://www.example.com
when: manual
only:
  - main
rules:
  - if: $CI_COMMIT_BRANCH == "main"

post_deploy_checks:
  stage: post-deploy
  image: alpine:3.18
  script:
    - apk add --no-cache curl
    - echo "Run basic healthcheck"
    - curl -fs https://staging.example.com/health || exit 1
  rules:
    - if: $CI_COMMIT_BRANCH == "develop" || $CI_COMMIT_BRANCH == "main"
```

Ghi chú:

- Thay frontend, backend, infra paths theo repo.
- KUBE_CONFIG_* là biến CI chứa kubeconfig base64 (protected + masked).
- Sử dụng Helm để deploy; có thể thay bằng kubectl apply cho manifest thuần.

10. Mẹo tối ưu và debug

- Local test pipeline: dùng gitlab-runner exec docker <job> để debug job local.
- Tối ưu cache: cache theo key branch + checksum của package-lock.json để invalidate đúng lúc.
- Tách pipeline: monorepo có thể dùng workflow:rules để chạy job chỉ khi file trong folder tương ứng thay đổi.
- Quản lý flaky tests: đánh dấu flaky và loại bỏ khỏi pipeline bắt buộc; phân tách e2e sang pipeline nightly.

11. Quản lý releases và rollback

- Release process: tạo tag semver (v1.2.0) → trigger release job. Dùng git tag + CI job only: tags để làm release.
- Rollback: lưu các manifest/helm values per release; dùng helm rollback hoặc deploy lại tag cũ.

- DB migrations: chạy migrations có kiểm soát (job riêng), thực hiện backup trước migration critical.

12. Bảo mật và scans tích hợp

- Thêm SAST/DAST/Dependency scanning jobs (GitLab built-in scanners) vào pipeline trước merge.
- Block merge: cấu hình Merge Request rules để bắt buộc security scan pass hoặc có exception process.
- Container image scan: scan image after build using tools (Trivy/Snyk) và fail pipeline nếu phát hiện vulnerability cao.

13. Checklist triển khai nhanh

1. Đăng ký runner (docker/k8s) và gán tag.
2. Tạo CI/CD Variables: registry creds, kubeconfigs, secrets (protected+masked).
3. Tạo .gitlab-ci.yml mẫu và push; kiểm tra pipeline run.
4. Bật Container Registry cho project.
5. Thiết lập branch protection cho main/release.
6. Configure Merge Request approvals & required pipeline pass.
7. Cấu hình review apps nếu muốn preview tự động.
8. Bật security scanners trong project settings.
9. Document pipeline trong repo README (how to run locally, troubleshooting).